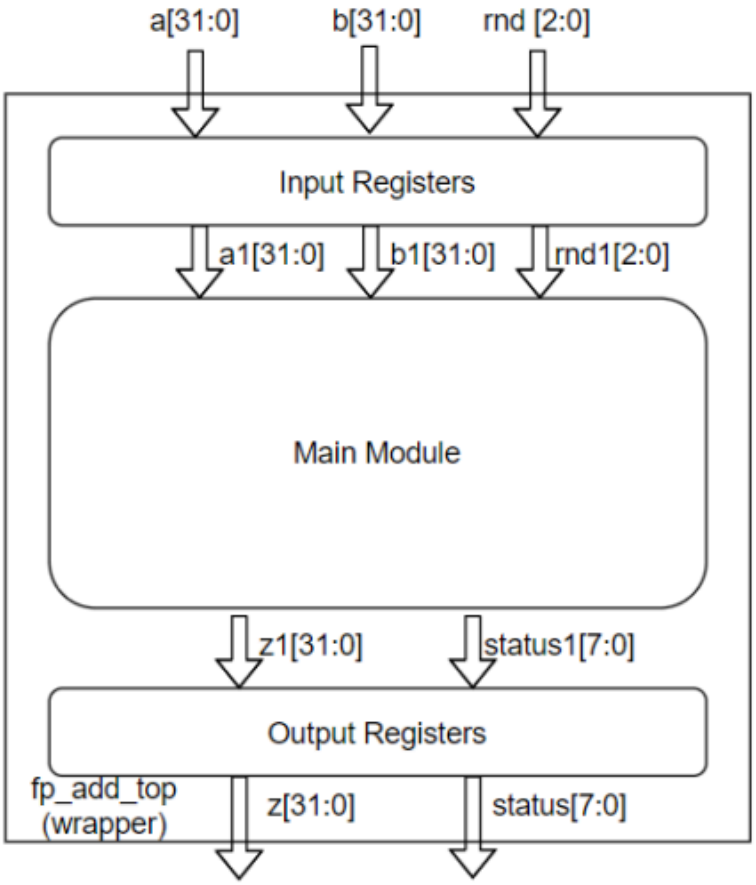
ARISTOTLE
UNIVERSITY
OF THESSALONIKI

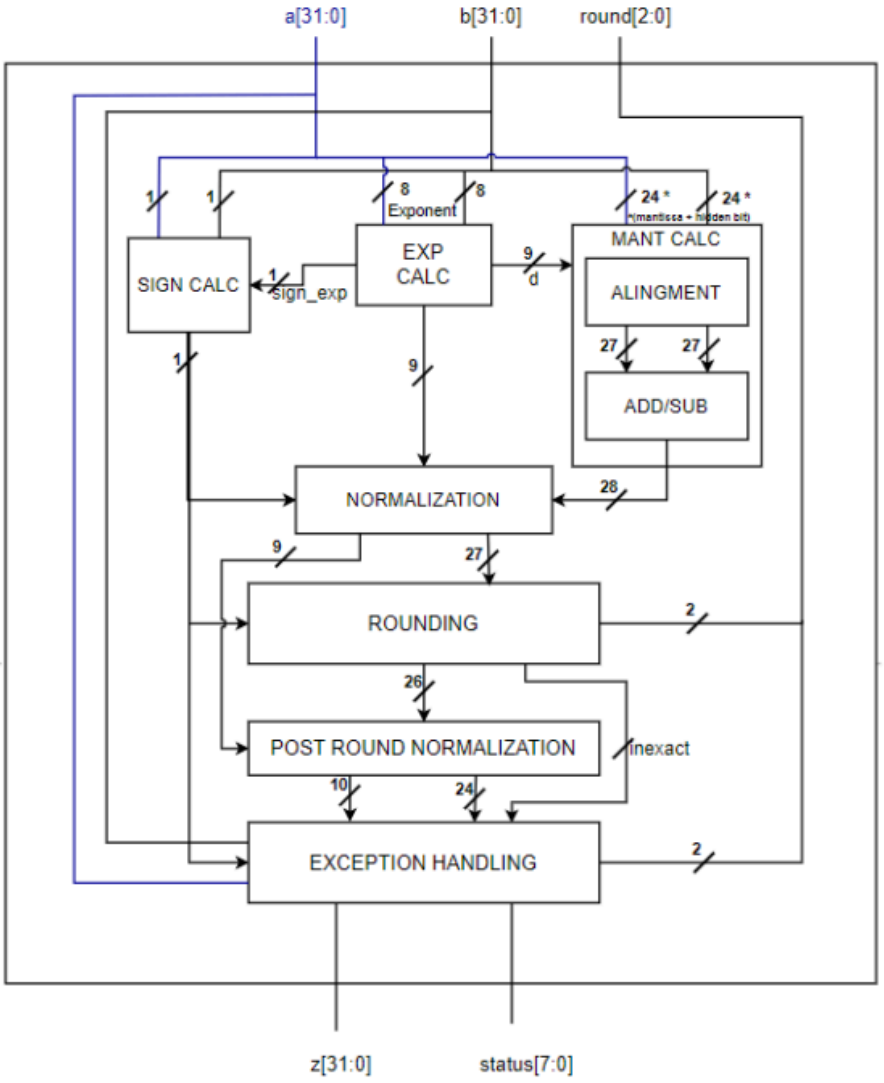
IEEE-754 Floating Point Adder Unit in System Verilog

Nikolaos Karapatsias 10661
Department of Electrical & Computer Engineering



1.Introduction

This document provides the report of a floating point adder unit fully implemented and verified using system verilog following the IEEE-754 standard. This project is made during the course Low-Level HW Digital Systems II of the spring semester. The main goal is to implement a 32bit functional floating adder point unit with 3 inputs and 2 outputs, as shown in the below block diagram.

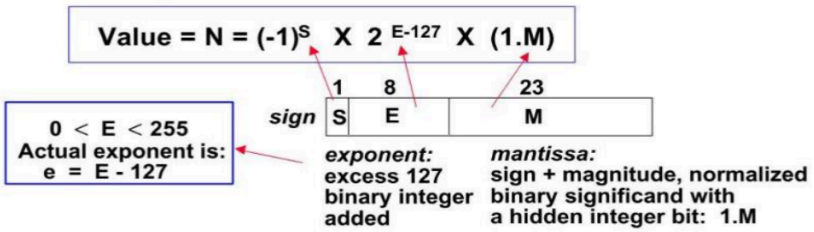


P 1.1 Project's block diagram

Ports description:

➔ Inputs

- **a and b inputs** are the two float 32 bit adders in the format of "$(-1)^{s} * 2^{E-127} * (1.M)$" where the single sign bit S, is either 0 or 1 if the number is positive or negative correspondingly, 23 bit mantissa M is the fractional portion of the number and an 8 bit exponent 2^{E} with which mantissa is multiplied to give the final number. A visual representation of the number is shown below in P 1.2



P 1.2 Float number binary representation IEEE-754 standard

- **Round 3 bit rnd** input which sets the type of rounding applied in the final result of the adder. Its different set modes are shown in the below table P 1.3

Round Type	round [3:0]	Description
IEEE_near	3'b000	Round to the nearest value. If equal near, round to the nearest even number
IEEE_zero	3'b001	Round towards zero
IEEE_ninf	3'b010	Round to -Infinity
IEEE_pinf	3'b011	Round to +Infinity
near_maxMag	3'b100	Round to the nearest value. If equal near round to the number with the maximum magnitude

P 1.3 Rounding types and description

➔ Outputs

- **The 32 bit result output Z** which holds the resulting value of float addition between the inputs a and b.
- **The 8 bit status register**. Each bit of it is a specific flag showing all the exception cases of the resulted number. Its detailed function is shown in the below table P 1.4

Bit	Flag	Description
0	Zero	$z = \pm Zero$
1	Infinity	$z = \pm Inf$
2	Invalid	$F = NaN$
3	Tiny	$\|F\| \neq Zero/Inf/NaN$ and $\|F\| < MinNorm$
4	Huge	$\|F\| \neq \pm Inf/NaN$ and $\|F\| > MaxNorm$
5	Inexact	$\|F\| \neq Zero/Inf/NaN$ and $F \neq z$
6	Unused	Reserved to 0
7	Div by 0	Division by zero, reserved to 0 otherwise

P 1.4 Meaning of each bit of the output status flag

2. Design Implementation

In this section, we are presenting our design approach of the floating point adder unit using system verilog. Note that our design consists of 8 main modules/parts as shown in the P 1.1 Each module is described separately in the following section.

❖ Fp adder

- ➢ Ports: Contains all the required ports of our adder hence it takes the two 32 bit adders a and b, the 3 bit round bits that allows to choose the type of round and produces the 32 bit adder result along with the 8 bit status register.

- ➢ Design: This is the top module of our project containing all the wiring between the different modules following the connections shown in P 1.1. This module is instantiated inside the fp_adder_top module/wrapper.

❖ Sign calculator

- ➢ Ports: This module has two single bit inputs, the *Most Significant Bit* (MSB) of each adder a and b and one “internal” *output z_out* holding the value 0 if the resulting number will be positive or 1 if negative.

- ➢ Design: The design of such module is really simple, implemented inside the fp_adder.sv file in an always_comb module and executing successively checks between: a) the signs of both inputs b) the exponents of each input c) the mantissas of each input d) sets a default positive value in case all are equal.

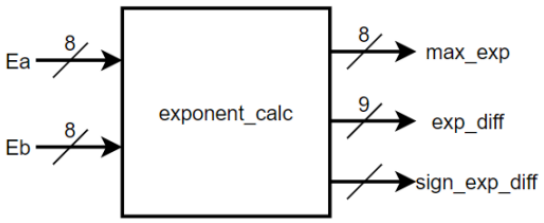
```
always_comb begin

   if(a1[31] == b1[31]) z_out[31] = a1[31]; // Checking sign bits
   else if (a1[30:23] > b1[30:23]) z_out[31] = a1[31]; // Checking exponent
   else if (a1[30:23] < b1[30:23]) z_out[31] = b1[31];
   else if (a1[22:0] > b1[22:0]) z_out[31] = a1[31]; // Checking mantissa
   else if (a1[22:0] < b1[22:0]) z_out[31] = b1[31];
   else z_out[31] = 1'b0; // Setting a deault value

end
```

P 2.1 Implementation of sign bit logic

❖ Exponent calculator module

➢ Ports: This module has two 8 bit inputs being the *exponent part* (bits 23 to 30) of the two adders a and b. Also it has 3 outputs resulting in the maximum exponential *max_exp* between the two inputs, the exponential difference between the maximum and the minimum exponents *exp_diff* and the *sign_exp_diff* flag which is set if the exponent of a is bigger than the exponent of b. Its block diagram is shown in P 2.2

➢ Design: The design implementation is again a simple exponent comparison between the two inputs implemented inside an always_comb block, since we are only using combinational logic. If the exponent of a *Ea,* is bigger than the exponent of b *Eb,* then we set the sign_exp_diff to 1, their difference exp_diff to Ea-Eb and mark the Ea as the bigger exponential through max_exp. Otherwise the results are the opposite as shown in the code snippet P 2.3 Last but not least we are setting the first case as the default one in case both are equal.



P 2.2 Block diagram of exponent_calc module

```
always_comb begin
   if (Ea[7:0]>=Eb[7:0]) begin // Setup a deault value in case they are equal
      sign_exp_diff = 1'b1;
      exp_diff = {1'b0, Ea[7:0]-Eb[7:0]};
      max_exp = Ea[7:0];
   end
   else begin
      sign_exp_diff = 1'b0;
      exp_diff = {1'b0, Eb[7:0]-Ea[7:0]};
      max_exp = Eb[7:0];
   end
end
```

P 2.3 Block diagram of exponent_calc module

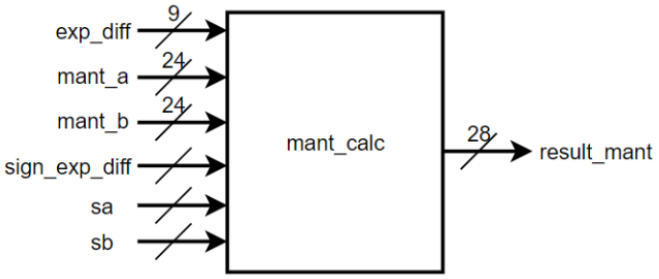
❖ Mantissa calculator module

➢ Ports: This module has multiple inputs. Firstly it receives the 9 bit exp_diff and the single but sign_exp_diff both calculated previously on the exponent_calc module. Also it receives the two 23 bit mantissas *mant_a* and *mant_b* from the two input numbers including one implicit-hidden bit as the MSB. Last but not least it receives the sign bits sa and sb of both a and b inputs, both featured in the MSB part of their corresponding number. A block diagram of mant_calc is shown in P 2.4

➢ Design: This module is responsible for the calculation of the initial/pre-normalization mantissa of the resulting number z. Firstly we are choosing the biggest and smallest mantissa based on the result of sign_exp_diff received, while padding it with three zeroes in the lowest part to form the 27 bit number required as shown in the P 2.5. Note that we reserve a 49 bit buffer *shift_capture* to safely hold the shift result.

Then we check if the difference between their exponents is bigger than 48 positions as shown in P 2.6. If so ,and also if the smaller mantissa is a non zero number, then we proceed to set the sticky bit to 1 and place it in the LSB place of the final mantissa result, indicating overflow situation. If the required shift exp_diff is smaller than 48 positions, as shown in picture P 2.7 then we proceed to shift the smaller mantissa by exp_diff amount of times (after adding 22 zeroes in its lowest part to form the 49 bit buffer properly) and then check if a 1 has fallen into the grs bits area during the shift process and set the sticky but correspondingly (0 if not) and then calculating the final aligned smaller mantissa.

Last but not least we are calculating the final mantissa by checking their signs. If both numbers have the same sign then we proceed to addition, otherwise we perform subtraction as shown in picture P 2.8.



P 2.4 Block diagram of mant_calc module

```
// Choosing the bigggest and smallest mantissas
if (sign_exp_diff == 1'b1) begin
   larger_mant  = {mant_a, 3'b000};
   smaller_mant = {mant_b, 3'b000};
end else begin
   larger_mant  = {mant_b, 3'b000};
   smaller_mant = {mant_a, 3'b000};
end
```

P 2.5 Choosing the smaller and bigger mantissas

```
if (exp_diff >= 9'd48) begin // If d is bi
   shift_capture = '0;
   sticky_bit = (smaller_mant != 0);
   shifted_smaller = {26'b0, sticky_bit};
end
```

P 2.6 Checking for overflow

```
else begin
   shift_capture = {smaller_mant, 22'b0} >> exp_diff; // Perform the shift o
   sticky_bit = |shift_capture[21:0];
   shifted_smaller = {shift_capture[48:23], shift_capture[22] | sticky_bit};
end
```

P 2.7 Normal operation

```
if (sa == sb) begin
   result_mant = larger_mant + shifted_smaller;
end

else begin
   result_mant = larger_mant - shifted_smaller;
end
```

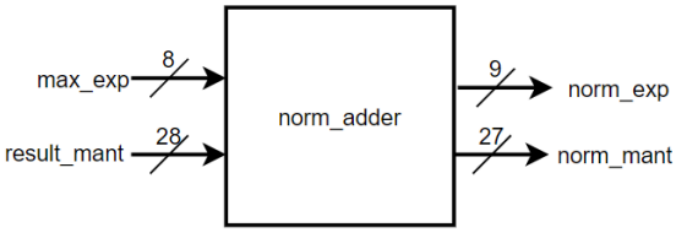
P 2.8 Final mantissa calculation

❖ Normalization module

- ➢ Ports: This module has two inputs and two output ports. The 8bit max_exp received from exp_calc module and the 28bit result_mant received from mant_calc module correspond to the input ports while the module itself produces the 9 bit norm_exp and 27 bit norm_mant which are respectively the final resulted normalized exponential and mantissa following the IEEE-754 standard.

- ➢ Design: This module is responsible to produce the final version of the mantissa and exponential. Based on the format of result_mant received from the mantissa calculator we proceed to the following three cases in order to comply with the IEEE-754 standard which obligates mantissa in format of 1.xxx:

- Case where mantissa is already in the right format 1.xxx: Mantissa is already in the right format hence we do not modify anything (P 2.10)
- Case where mantissa is equal or greater than two 1x.xxx: In that case we need to right shift the received mantissa and increase by one the exponential (P 2.11)
- Case where mantissa is smaller than one 0.zz…xxx: In that case we need to transform the received mantissa into the right format 1.xxx. To achieve that we firstly create another module named leading_zero_counter which counts the number of zeroes before the actual value (denoted as z above). Then we left shift the mantissa until the leading bit is equal to one while also subtracting the amount of shifts we executed from the exponential portion to produce the final norm_exp (P 2.12)



P 2.9 Block diagram of norm_adder module

```
norm_mant = result_mant[26:0];
norm_exp = {1'b0, max_exp};
```

P 2.10 Case where mantissa is in 1.xxx format

```
norm_mant = {result_mant[27:2], result_mant[1] | result_mant[0]};
norm_exp = max_exp + 1;
```

P 2.11 Case where mantissa is in 1x.xxx format

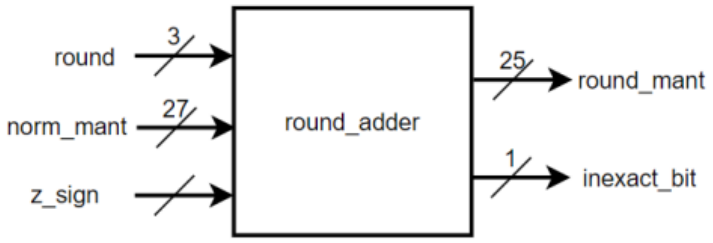
```
norm_mant = result_mant[26:0] << leading_zeroes;
norm_exp = max_exp-leading_zeroes;
```

P 2.12 Case where mantissa is in 0.zzz…xxx format

❖ Rounding module

- ➢ Ports: This module has three inputs and two output ports. The 3 bit round received from the top module input and dictates the type of rounding we will follow based on the table P 2.14, 27 bit norm_mant mantissa ,including grs bits, received from the normalization module and z_sign calculated on the sign calculator part previously. Then it produces the 25 bit round_mantissa and the inexact_bit which shows if rounding applied.

- ➢ Design: This module is responsible to perform rounding and produce the final mantissa form. Firstly we are creating a package named round_pkg which help us visualize the round type chosen based on P 2.14 decoding the binary format. Is operation is shown in P 2.15. Then we proceed to the main part of this module by applying the round type chosen from the round bits using a case block shown in P 2.16. Firstly we check for rounding to the next nearest value which is taken only if the guard bit is set and either round or sticky bit is one. In case we are exactly in the middle, so guard is one and the others are zero, we round to the nearest even number which is indicated by the LSB. If this bit is one then the current number is odd hence we need to round to the next one, if not then r|s|mant_LSB are zero indicating that we are already in an even number thus we don't need to change anything. Then we check the round towards zero case in which we completely ignore the grs bits and dont proceed to any rounding. Then following the round towards negative infinity in which if there is any leftover (one of grs bits is set) and the number is negative then we apply rounding by which we rise the magnitude away from zero. The same happens with round towards positive infinity in case the number is positive. Last but not least we check for Round to Nearest in which we apply rounding any the case where g bit is set even if the number is right in the middle. Then after all those checks we proceed to apply the round portion as shown in P 2.17.



P 2.13 Block diagram of round_adder module

Round Type	round [3:0]	Description
IEEE_near	3'b000	Round to the nearest value. If equal near, round to the nearest even number
IEEE_zero	3'b001	Round towards zero
IEEE_ninf	3'b010	Round to -Infinity
IEEE_pinf	3'b011	Round to +Infinity
near_maxMag	3'b100	Round to the nearest value. If equal near round to the number with the maximum magnitude

P 2.14 Rounding types and description

```
localparam logic [2:0] IEEE_near = 3'b000;
localparam logic [2:0] IEEE_zero = 3'b001;
localparam logic [2:0] IEEE_ninf = 3'b010;
localparam logic [2:0] IEEE_pinf  = 3'b011;
localparam logic [2:0] near_maxMag = 3'b100;
```

P 2.15 round package snippet

```
case (round)
    IEEE_near: begin
        round_up = g & (r | s | base_mant[0]);   // Round to nearest value
    end

    IEEE_zero: begin
        round_up = 1'b0;    // Round towards zero
    end

    IEEE_ninf: begin
        round_up = inexact_bit & z_sign;   // Round to -Inf
    end

    IEEE_pinf: begin
        round_up = inexact_bit & ~z_sign;  // Round to +Inf
    end

    near_maxMag: begin
        round_up = g; // Round near_maxMag
    end

    default: begin
        round_up = 1'b0;
    end
endcase
```

P 2.16 round package snippet

```
assign round_mant = {1'b0, base_mant} + round_up; // Perform rounding
```

P 2.17 Apply rounding

❖ Post-round normalization module

- ➢ Ports: This logic is implemented again inside the top module (fp_adder.sv) hence it doesn't have independent ports. It receives the 25 bit mantissa, including the overflow/underflow detection bit, the exponential and the calculated sign bit and produces the final number -assuming no overflow/underflow- along with overflow and underflow flags indicator

- ➢ Design: Firstly we check if the MSB/hidden bit of the calculated mantissa indicates overflow. If it does then we shift back the mantissa by one and adjust correspondingly the exponent by adding one. If overflow didn't occur then we simply get rid of the overflow indicator bit/MSB as shown in P 2.19. Then we proceed to set the overflow and underflow flags by doing the checks shown in P 2.20. Last but not least we are forming the final version of the resulted number as shown in the P 2.21 based on the checks of P 2.19

```
if (round_mant[24] == 1'b1) begin
   post_round_exp = norm_exp + 1;
   post_round_frac = round_mant[23:1];
end else begin
   post_round_exp = norm_exp;
   post_round_frac = round_mant[22:0];
end
```

P 2.19 Calculate the post-norm mantissa and exp

```
// Overflow: If exponent reaches 255
if (post_round_exp >= 9'd255 && post_round_exp[8] == 1'b0) begin
   overflow = 1'b1;
end else begin
   overflow = 1'b0;
end

// Underflow: If exponent is 0 or less.
if (post_round_exp == 9'd0 || post_round_exp[8] == 1'b1) begin
   underflow = 1'b1;
end else begin
   underflow = 1'b0;
end
```

P 2.20 Check if overflow

```
z_calc = {z_out[31], post_round_exp[7:0], post_round_frac};
```

P 2.21 Produce the final normalized result

❖ Exception handling module

- ➢ Ports: Receives the 32-bit operands a and b, the 3-bit round mode, the z_calc intermediate result from the post-round normalization stage, and the generated overflow, underflow, and inexact_bit flags. In turn, it produces the final 32-bit result and the 6 status-bit flags (zero_f, inf_f, nan_f, tiny_f, huge_f, inexact_f) which will be routed to the final status register.

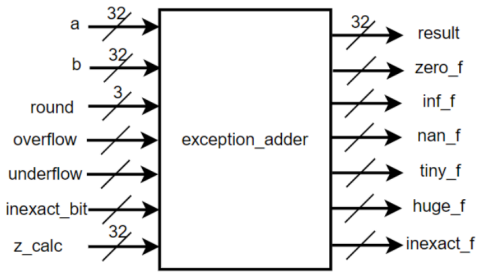
- ➢ Design: This module is responsible for handling all corner cases. Firstly, an enumerated type interp_t is defined, as required, to categorize numbers into distinct groups (ZERO, INF, NORM, MIN_NORM, MAX_NORM). Two internal functions are created to assist with the logic (P 2.23):

    - **num_interp**: Evaluates the 8-bit exponent of the input operands. It flushes denormals to ZERO, treats NaNs and Infinities as INF, and classifies everything else as NORM.

    - **z_num**: A generator function that quickly constructs and returns the 31-bit unsigned body (exponent and mantissa) for special boundary values like Infinity or Max Normal, making the main code cleaner.

Inside an always_comb block, all status flags are initialized to 0 and the result is defaulted to z_calc. Then, the module checks the interpreted types of operand a and b. If both are ZERO, it forces a zero output and asserts zero_f. If one is ZERO and the other is INF, it asserts inf_f and outputs infinity. A critical check happens when both inputs are INF; if their signs are different (e.g., +Inf and -Inf), the operation is mathematically invalid, so the module constructs a standard qNaN (32'h7FC00000) and sets the nan_f flag. (P 2.24)

Then we proceed to Normal x Normal Combinations. If the inputs do not match with the corner cases above, the module looks at the overflow and underflow flags generated in the previous stage. If the exponent grew too large (Overflow), huge_f and inexact_f are set. A case statement evaluates the rounding mode and the sign of the result to clamp the output to either exact Infinity or the Maximum Normal (MAX_NORM) number. If the exponent dropped to zero or below (Underflow), tiny_f and inexact_f are set. Similarly, the rounding mode and sign are evaluated to clamp the output to either exact ZERO or the Minimum Normal (MIN_NORM) number. Last but not least, If neither flag is triggered, the module safely passes z_calc to the final result and propagates the inexact_bit.



P 2.22 Block diagram of exception_adder module

```
function interp_t num_interp(input logic [31:0] num);
    if (num[30:23] == 8'h00)
        return ZERO;
    else if (num[30:23] == 8'hFF)
        return INF;
    else
        return NORM;
endfunction

function logic [30:0] z_num(input interp_t type_val);
    case (type_val)
        ZERO:     return 31'h00000000;
        INF:      return {8'hFF, 23'h000000};
        MIN_NORM: return {8'h01, 23'h000000};
        MAX_NORM: return {8'hFE, 23'h7FFFFF};
        default:  return 31'h00000000;
    endcase
endfunction
```

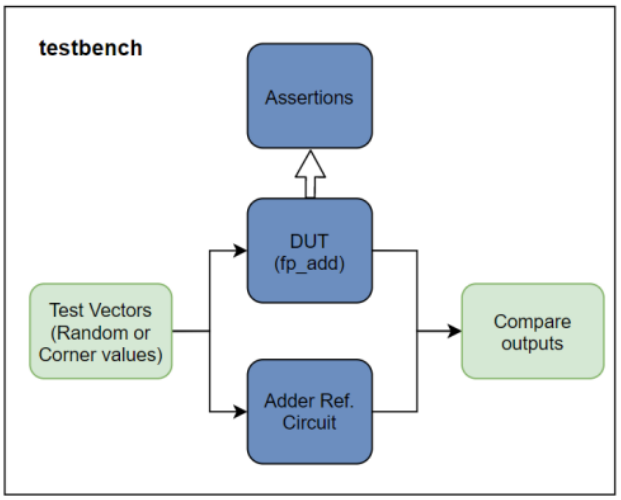
P 2.23 Implementation of num_interp and z_num

$(-1)^{SA}\ 2^{EA}\ 1.MA$	$(-1)^{SB}\ 2^{EB}\ 1.MB$	$(-1)^{SZ}\ 2^{EZ}\ 1.MZ$
+/- ZERO	+/- ZERO	+/- ZERO
+/- ZERO	+/- INF	+/- INF
+/- ZERO	+/- NORM	+/- NORM
+INF	-INF	NAN
+ /- INF	+/- NORM	+/- INF

P 2.24 Possible corner case combinations for a and b signals. Combination Norm x Norm is not included in the table

3. Design Verification

In this third and final part, we are going to proceed to the verification stage of our design. Our goal is to create a testbench written in again in SystemVerilog aiming to generate a test-vector with random and corner values and then compare the results with a reference model. Last but not least, systemverilog's assertions will be showcased to prove the correct function of our design (P 3.1)



P 3.1 Testbench block diagram structure

Implementation

Firstly we are setting up the variables needed both for the DUT and the reference designs to feed the inputs and store the produced results. We are then proceeding to implement a 2 cycle delay logic since the designed floating adder needs 2 cycles to produce the results while the reference produces them instantly, hence we create an artificial "pipeline" for that reason as shown in P 3.2

```
always_ff @(posedge clk or negedge resetn) begin
  if (!resetn) begin
       is_random_d1 <= 0;
       is_random_d2 <= 0;
       valid_check_d1 <= 0;
       valid_check_d2 <= 0;
       ref_delay1 <= '0;
       ref_delay2 <= '0;
  end else begin
       is_random_d1 <= is_random;
       is_random_d2 <= is_random_d1;
       valid_check_d1 <= valid_check;
       valid_check_d2 <= valid_check_d1;
       ref_delay1 <= results_ref;
       ref_delay2 <= ref_delay1;
  end
end
```

P 3.2 2 cycle delay

We then proceed to instantiate the two designs both DUT and reference and dismiss the inputs that our DUT is not made to recognize by changing their type. More specifically, if one of the inputs is a denormal we convert it to zero, or if is NaN to Inf as shown in P 3.3.

```
// If a is NaN => Inf
if(a[30:23] == '1) begin
   a_hf = {a[31], {8{1'b1}}, {23{1'b0}}};
end
// If a is denorm => Zero
else if(a[30:23] == '0 ) begin
   a_hf = {a[31], {31{1'b0}}};
end
else begin
   a_hf = a;
end
```

P 3.3 Reshape the data if needed

We then proceed to create two functions to encode and decode the corner cases with an enumeration struct hence easies readability as shown in P 3.4

```
function logic [31:0] get_corner_value(corner_case_t corner);
  case (corner)
      pos_nan:    return 32'h7FC00000;
      neg_nan:    return 32'hFFC00000;
      pos_inf:    return 32'h7F800000;
      neg_inf:    return 32'hFF800000;
      pos_norm:   return 32'h40000000;
      neg_norm:   return 32'hC0000000;
      pos_denorm: return 32'h00400000;
      neg_denorm: return 32'h80400000;
      pos_zero:   return 32'h00000000;
      neg_zero:   return 32'h80000000;
      default:    return 32'h00000000;
  endcase
endfunction
```

P 3.4 Corner case enum generator function

Then we are configuring two main tasks named **run_corner_cases()** and **run_random_cases()**. Firstly we are looping all the possible corner cases (5) through all the possible rounding cases (10) for each input so 500 in total P3.5.

```
// Loop all 5 rounding modes
for (int r = 0; r < 5; r++) begin
    round = r[2:0];

    // Loop all 10 corner cases for input a
    for (int i = 0; i < 10; i++) begin
        // Loop all 10 corner cases for input b
        for (int j = 0; j < 10; j++) begin
            @(negedge clk);
            valid_check = 1;
            is_random = 0;
            a = get_corner_value(corner_case_t'(i));
            b = get_corner_value(corner_case_t'(j));
        end
    end
end
```

P 3.5 Corner case task run_corner_cases()

Then proceed with the same logic generating 1000 random values per rounding mode (5) hence 5000 in total P3.6

```
// 1000 random tests per rounding mode
for (int r = 0; r < 5; r++) begin
    round = r[2:0];

    for (int k = 0; k < 1000; k++) begin
        @(negedge clk);
        valid_check = 1;
        is_random = 1;
        a = {$urandom(), $urandom()};
        b = {$urandom(), $urandom()};
    end
end
```

P 3.6 Random values task run_random_cases()

System Verilog Assertions

Before we proceed to verify our design we need to create assertions demonstrating the wide spectrum of system verilog's toolset inside the SVA.sv file. Firstly we are going to import Immediate Assertions to all possible cases to ensure that two bits can never be 1 at the same time in the status output register. More specifically, we are going to ensure that none of the below are set simultaneously:

a) NaN and Zero/Inf/Tiny/Huge
b) Zero and Huge
c) Inf and Tiny
d) Tiny and Huge

A snippet of the last one is shown in picture P 3.7. We followed the same logic for the rest.

```
if (status[3]) begin
    assert_tiny_huge: assert(status[4] == 0)
        $display("[PASS] Tiny and Huge did not assert together.");
        else $error("[FAIL] Tiny and Huge asserted together!");
end
```

P 3.7 Tiny and Huge bits assertion

We then proceed to create a concurrent assertion check as the requirements pdf direct. More specifically we are importing assertions for each of the following cases:

a) If 'zero' asserts, all bits of 'z' exponent must be 0
b) If 'inf' asserts, all bits of 'z' exponent must be 1.
c) If 'nan' asserts, 2 cycles before (due to our pipeline), 'a' and 'b' must have been infinite with opposite signs (+Inf and -Inf).
d) If 'huge' asserts, result exponent must be 0xFF (Inf) OR 0xFE with all 1s mantissa (MaxNormal).

A snippet of the last one is shown in picture P 3.8. We followed the same logic for the rest.

```
property p_huge;
    @(posedge clk) status[4] |-> (result[30:23] == 8'hFF) ||
                                 (result[30:23] == 8'hFE && result[22:0] == 23'h7FFFFF);
endproperty

assert_huge: assert property (p_huge)
    else $error("[FAIL] Huge status asserted without Inf or MaxNormal result.");

cover_huge: cover property (@(posedge clk) status[4] && ((result[30:23] == 8'hFF) || (result[30:23] == 8'hFE && result[22:0] == 23'h7FFFFF)))
    $display("[PASS] Huge flag successfully matched with Overflow boundaries.");
```

P 3.8 Huge and result exponent.mantissa assertion

Last but not least we proceed to bind those assertions into our main testbench as depicted in P 3.9.

```
// Bind the Assertions to the DUT
bind fp_adder_top test_status_bits dut_status_bits_inst (.status(status));
bind fp_adder_top test_status_z_combinations dut_status_z_inst (.clk(clk), .a(a), .b(b), .result(result), .status(status));
```

P 3.8 Bind the assertions

Results

Finally we proceed to initialize our main clock with 10ns period, call our two main tasks and then evaluate the results and compare the differences between our DUT and the reference model. The results are shown below in P 3.9

```
Total Random Tests Executed : 5000
Random Tests SUCCESS Rate   : 4982 / 5000
------------------------------------------------------
Total Corner Tests Executed : 500
Corner Tests SUCCESS Rate   : 418 / 500
```

P 3.9 Statistics Results

We notice that even though our results are pretty good, the two models are not matching perfectly. For the corner cases tests we notice that our coursework instructed us to treat all NaNs as Infinity numbers in the rounding module, as shown below in P 3.10

```
CORNER ERROR | Rnd: 4 | Expected: 7f800000 | Got: 7fc00000
```

P 3.10 NaN vs Infinity mismatch

Also we do notice some mismatches on the positive and negative zero P 3.11. This is probably due to lack of complex calculations on our sign calculator part of the design.

```
CORNER ERROR | Rnd: 3 | Expected: 00000000 | Got: 80000000
```

P 3.11 Expected positive but got negative zero

Also we do notice some mismatches on the positive and negative zero P 3.11. This is probably due to lack of complex calculations on our sign calculator part of the design. Those results led our DUT to 83.6% similarity in comparison with the reference model for corner cases tests and 99.6% for random values.

Last but not least we notice that there were no assertion violations and all of the checks were successfully PASSED! (P 3.12)

```
[PASS] Inf and Tiny did not assert together.
[PASS] Infinity flag successfully matched with 0xFF exponent.
```

P 3.12 Snippet of passed assertions